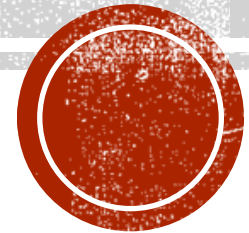




**Faculty of Economics and
Business Administration**

An introduction to flutter



Dr. Abdul Rahman EL SAYED

WHAT IS FLUTTER?

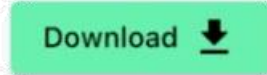
- ❑ Flutter is a collection of Widgets.
- ❑ Everything in Flutter is a Widget.
- ❑ Widgets are Dart classes that represent UI components such as Icons and Buttons.
- ❑ All Widgets inherit (extends) from either of 2 classes:
 - StatelessWidget class
 - StatefulWidget class



STATELESS WIDGETS?

- ❑ Stateless Widgets extends from the StatelessWidget class.
- ❑ They have a fixed (static) state that can't be changed at runtime. This means you can't change their contents, without re-creating the widgets.
- ❑ Below are some examples of Stateless Widgets:

- Text
- Icon
- Button



STATEFUL WIDGETS?

- ❑ Stateful Widgets extends from the StatefulWidget class.
- ❑ The Widget can change its state (contents) dynamically at runtime.
- ❑ Below are some examples of Stateful Widgets:

- CheckBox
- Radio
- TextField

Radio Buttons

Select your Subject ?

☐ DBMS

☐ C/C++ Programing

☐ Data Structure

☐ Algorithms

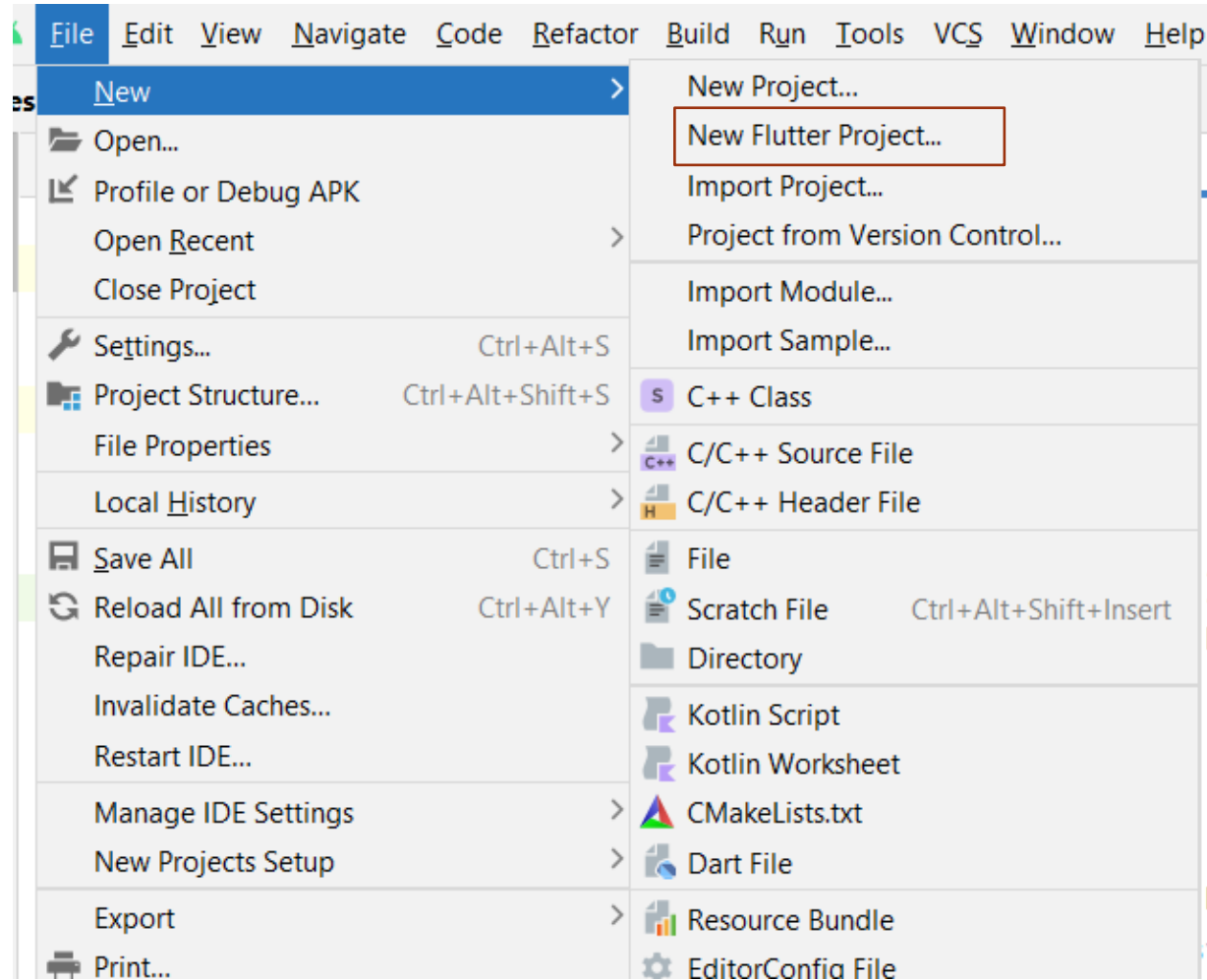
CLEAR SUBMIT

Radio Buttons in Radio Group



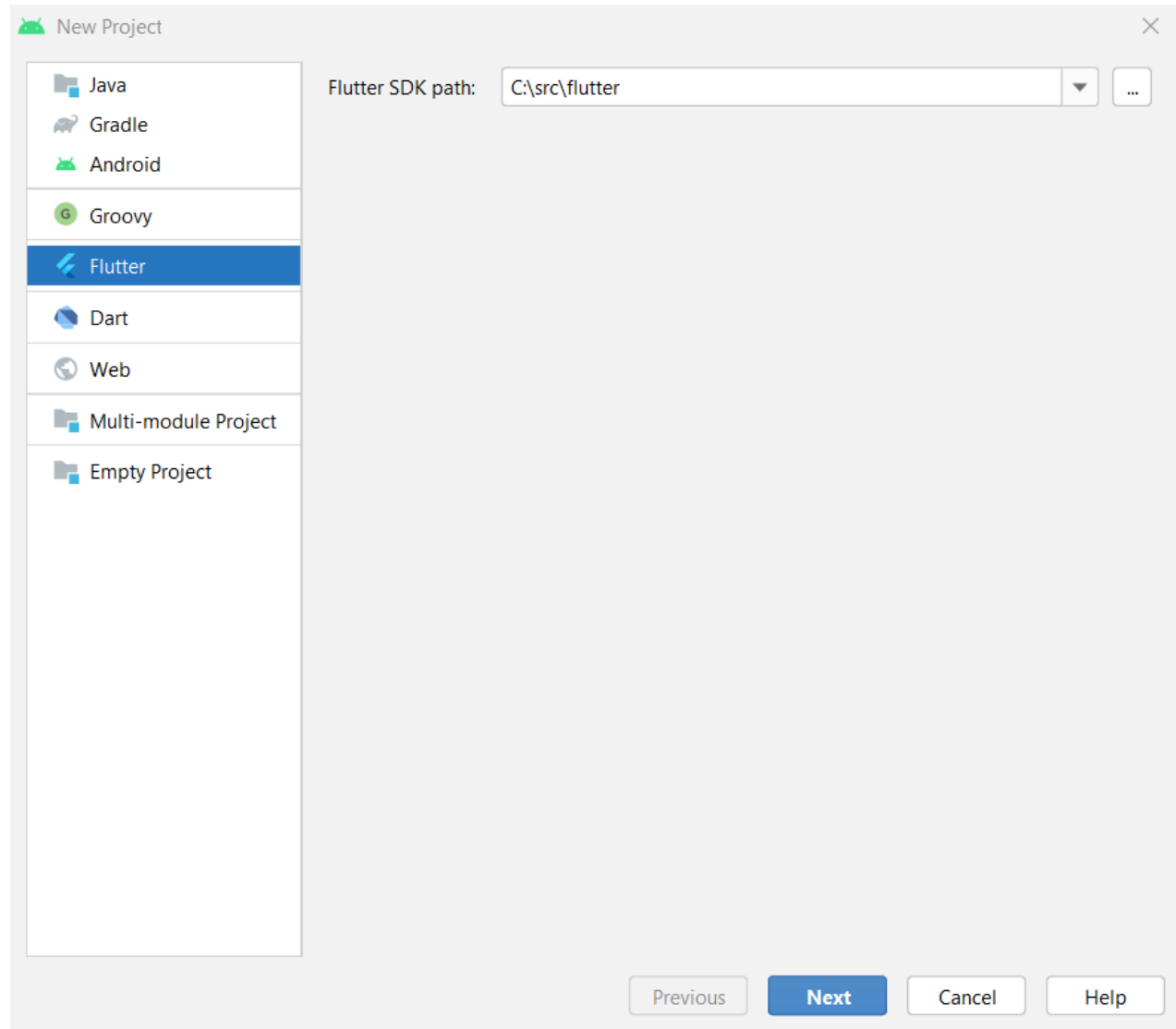
CREATING A FLUTTER PROJECT

We will create a simple hello world application. Start android studio and then click on New Flutter Project as shown below:



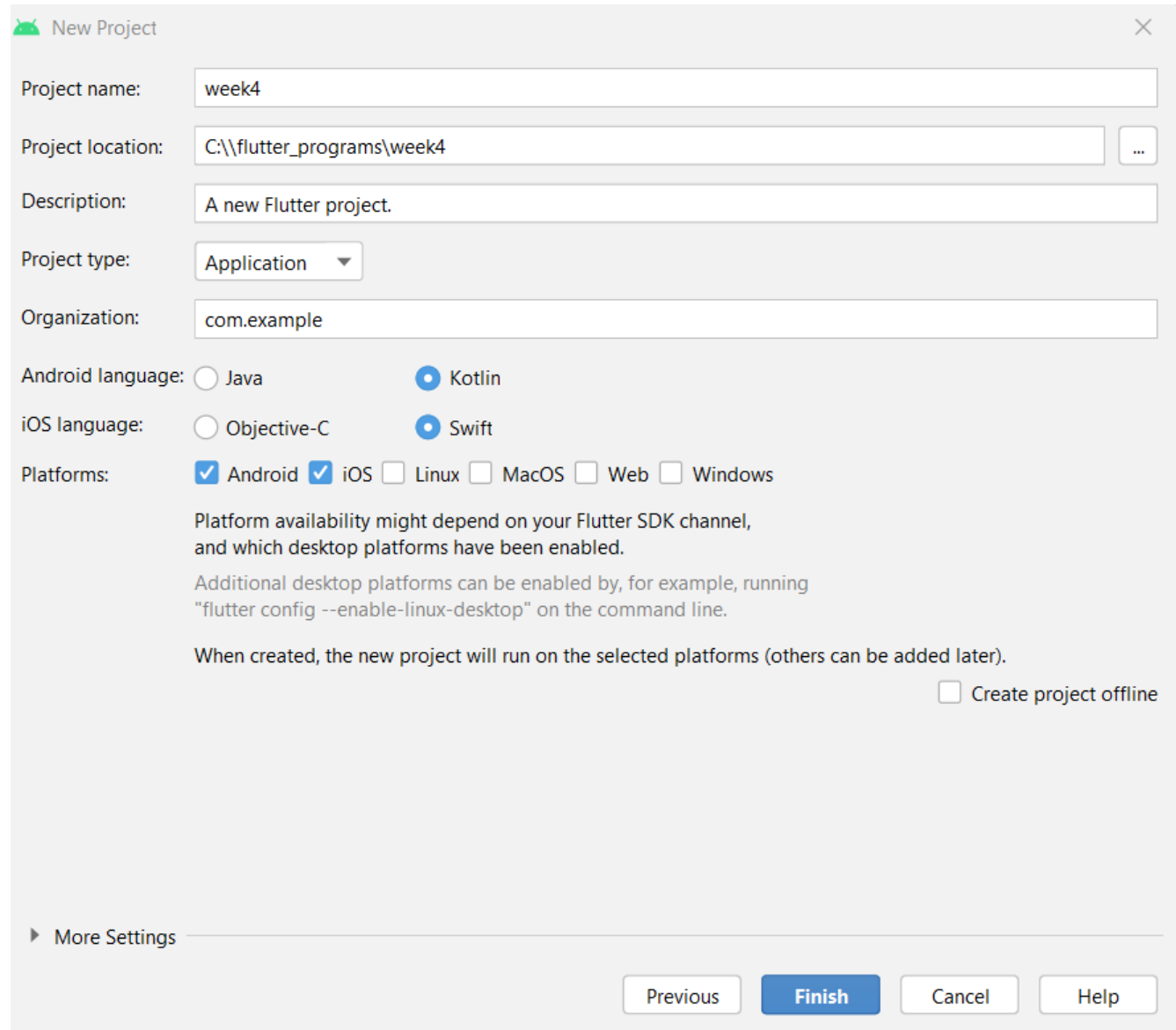
CREATING A FLUTTER PROJECT

Make sure the Flutter SDK path is correct and then click Next.



CREATING A FLUTTER PROJECT

Write week4 for the project name and select the folder were you want to create the project. Finally, click on Finish.

A screenshot of the 'New Project' dialog in an IDE. The dialog has a title bar with a Flutter logo and a close button. It contains several input fields and checkboxes. The 'Project name' field is filled with 'week4'. The 'Project location' field is filled with 'C:\\flutter_programs\\week4' and has a browse button. The 'Description' field is filled with 'A new Flutter project.'. The 'Project type' dropdown is set to 'Application'. The 'Organization' field is filled with 'com.example'. Under 'Android language', 'Kotlin' is selected. Under 'iOS language', 'Swift' is selected. Under 'Platforms', 'Android' and 'iOS' are checked. There is a 'More Settings' link at the bottom left. At the bottom right are buttons for 'Previous', 'Finish', 'Cancel', and 'Help'.

New Project

Project name: week4

Project location: C:\\flutter_programs\\week4 ...

Description: A new Flutter project.

Project type: Application

Organization: com.example

Android language: ☐ Java ☒ Kotlin

iOS language: ☐ Objective-C ☒ Swift

Platforms: ☒ Android ☒ iOS ☐ Linux ☐ MacOS ☐ Web ☐ Windows

Platform availability might depend on your Flutter SDK channel, and which desktop platforms have been enabled.

Additional desktop platforms can be enabled by, for example, running "flutter config --enable-linux-desktop" on the command line.

When created, the new project will run on the selected platforms (others can be added later).

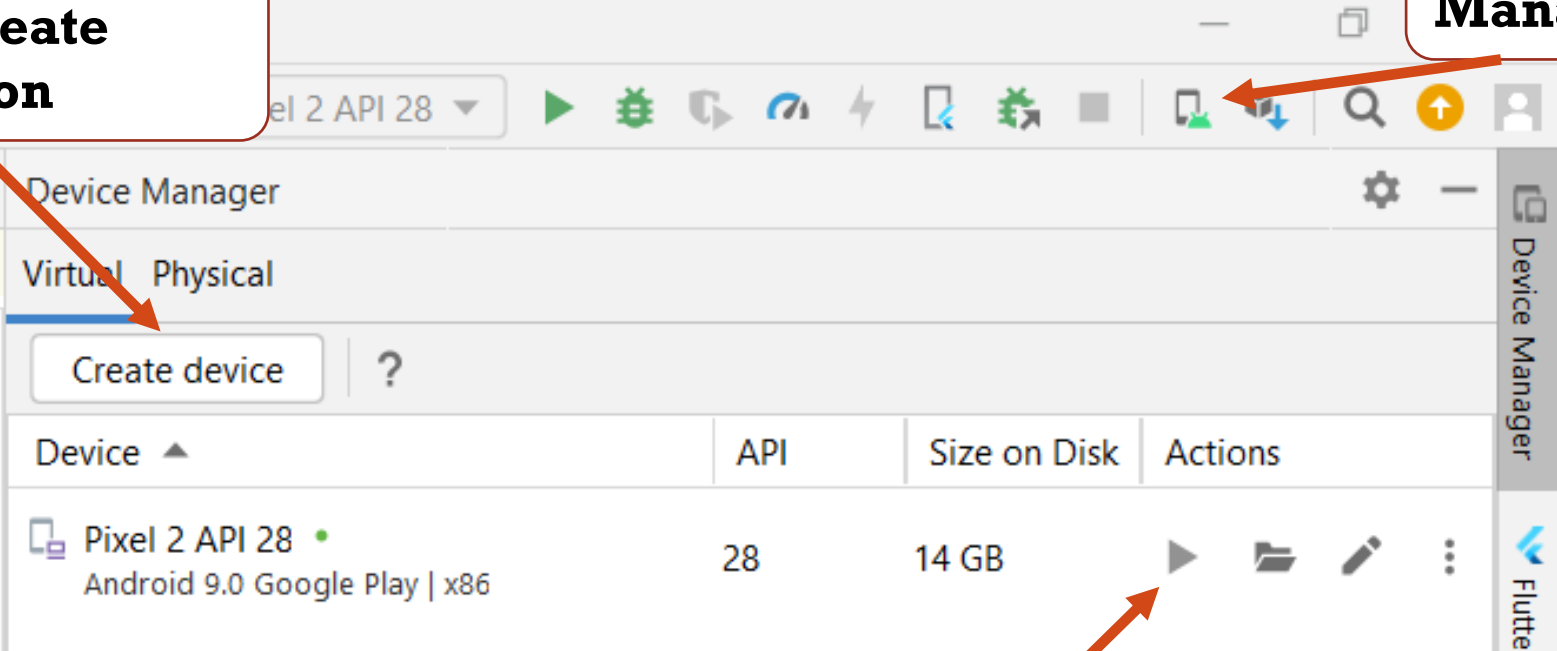
☐ Create project offline

► More Settings

Previous Finish Cancel Help

CREATING AN EMULATOR

Android Studio will create a new project with a sample application. You need to create a device emulator to test your project. You create a new emulator as shown below:



The screenshot shows the Android Studio interface with the Device Manager tab selected. The top toolbar contains various icons, including a green Android icon with a plus sign. The Device Manager panel has tabs for 'Virtual' and 'Physical', with 'Virtual' selected. Below the tabs is a 'Create device' button. A table lists the existing virtual device: 'Pixel 2 API 28' with API level 28 and a size of 14 GB. The 'Actions' column for this device contains a play button (launch icon), a folder icon, an edit icon, and a more options icon. Three red callout boxes with arrows point to specific elements: one to the 'Create device' button, one to the green Android icon in the toolbar, and one to the play button in the actions column.

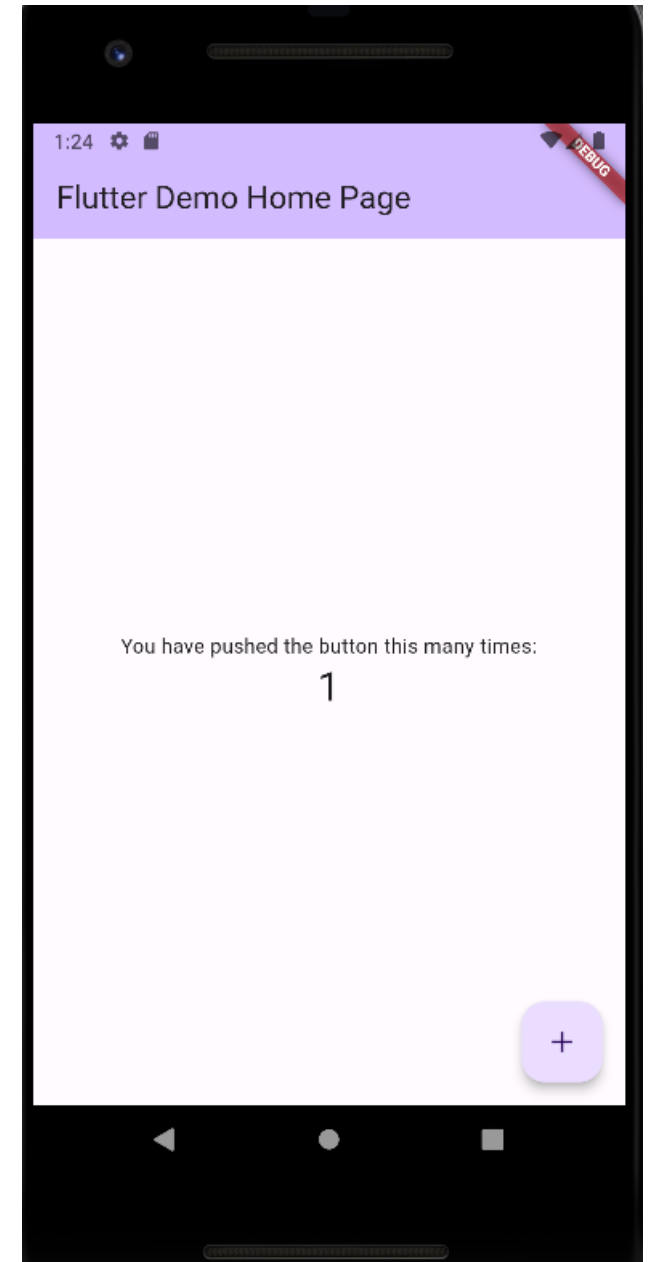
Click on Create device button

Click on the Device Manager Icon

After the device is created, click on the Launch icon.

RUNNING THE SAMPLE APPLICATION

- You can click on the Run Icon, to run your application and display it in the emulator.
- You can test the application by clicking on the add button, on the bottom left corner of the screen.



HELLO WORLD APPLICATION

Now we will create a hello world application. Remove all the code in the main.dart file and replace it with the below code:

```
import 'package:flutter/material.dart';

void main() => runApp(const MyApp());
// you can type st and then select stless, to create the below class.
class MyApp extends StatelessWidget {
  const MyApp({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Container();
  }
}
```



CODE EXPLAINED

```
import 'package:flutter/material.dart';
```

The material.dart package contains the widgets we need in this project.

```
void main() => runApp(const MyApp());
```

Since we are writing Dart code, we need to define a main method. In the above code we used the lambda notation, to call the runApp function.

The runApp function takes as parameter a widget and attaches it to the screen. By default, it will fill the entire screen. We passed to it the MyApp stateless widget. We used the const keyword to call the constructor, because we are using a stateless widget, its state will not change at run time. Using the const keyword makes your code run faster.



MYAPP STATELESS WIDGET

- ❑ You can create this widget by typing st in android studio and then selecting stless. You need to type the widget name “MyApp”.
- ❑ We will use a stateless widget only, since the contents of the screen will not change at runtime.
- ❑ The widget contains a build method that is responsible for returning a widget that will be displayed on the screen.
- ❑ By default, it will return an empty Container widget. When you run the application you will see an empty black screen. You can define a color for the Container, example white, as shown below:

```
class MyApp extends StatelessWidget {  
  const MyApp({Key? key}) : super(key: key);  
  
  @override  
  Widget build(BuildContext context) {  
    return Container(color: Colors.white);  
  }  
}
```



DISPLAYING TEXT USING THE TEXT WIDGET

Modify the Build method to return the below code:

```
class MyApp extends StatelessWidget {  
  const MyApp({Key? key}) : super(key: key);  
  
  @override  
  Widget build(BuildContext context) {  
    return MaterialApp(  
      home: Center(  
        child: Text('Hello World'),  
      ),  
    );  
  }  
}
```



CODE EXPLAINED

- ❑ In order to use the Text widget, we need to wrap it inside a MaterialApp class. This will give the text a default style (ex: color).
- ❑ The MaterialApp class has optional fields, one required field is “home”. This field defines the parent widget that will be displayed on the screen.
- ❑ By default the text will be displayed in the top left corner of the screen.
- ❑ We need to wrap the Text widget inside a Center widget to center the text on the screen.
- ❑ The center widget contains a required “child” field, this defines the widget that will be centered.



SAMPLE RUN

When you use the `MaterialApp` class alone to display the `Text` widget, it will display it with an ugly red/yellow text style that's intended to warn the developer that their app hasn't defined a default text style.

We will fix this by wrapping the `Text` widget inside a `Scaffold` class.

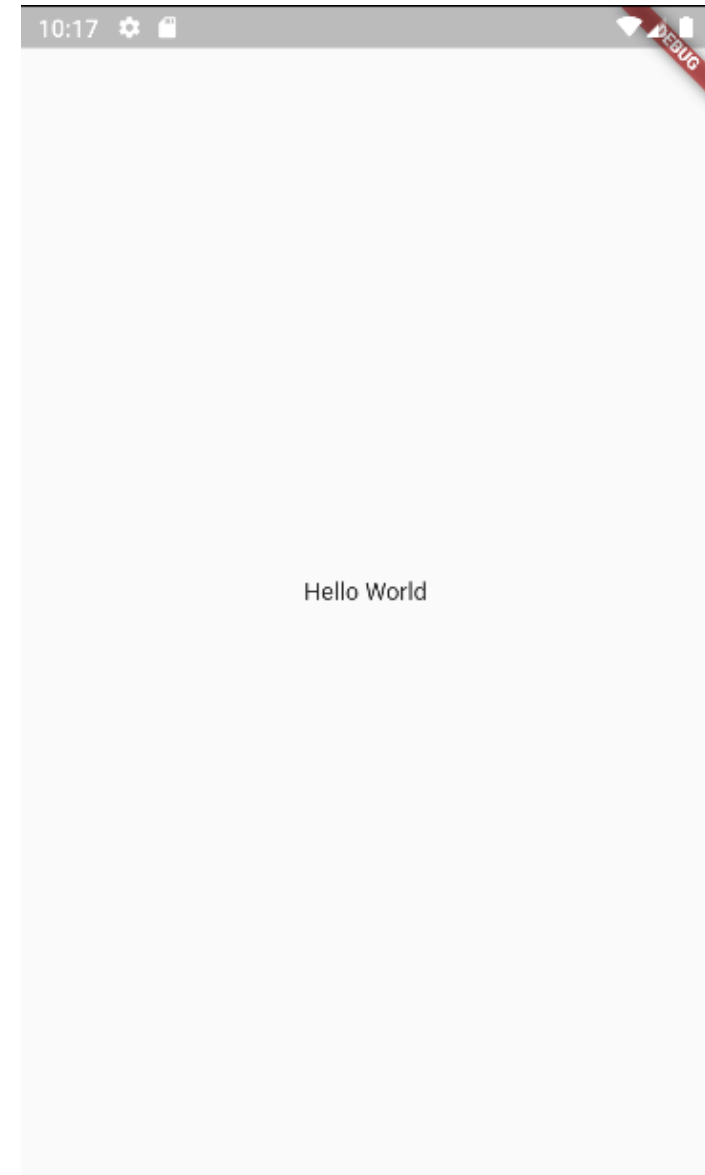


ADDING THE SCAFFOLD CLASS

We will wrap the Center and Text widgets inside a Scaffold class. This will give it a better looking background and text color.

We use the Scaffold's "body" field, to define the parent widget inside the Scaffold class.

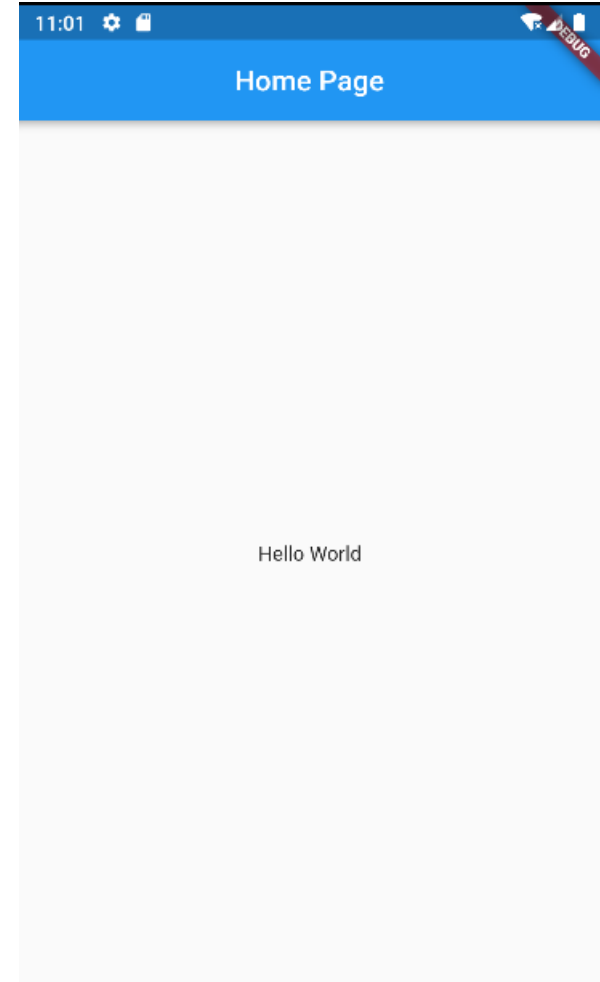
```
@override
Widget build(BuildContext context) {
  return MaterialApp(
    home: Scaffold(
      body: Center(
        child: Text('Hello World'),
      ),
    ),
  );
}
```



ADDING AN APPBAR

You can add an AppBar using the Scaffold's "appBar" field. You pass to it an AppBar widget.

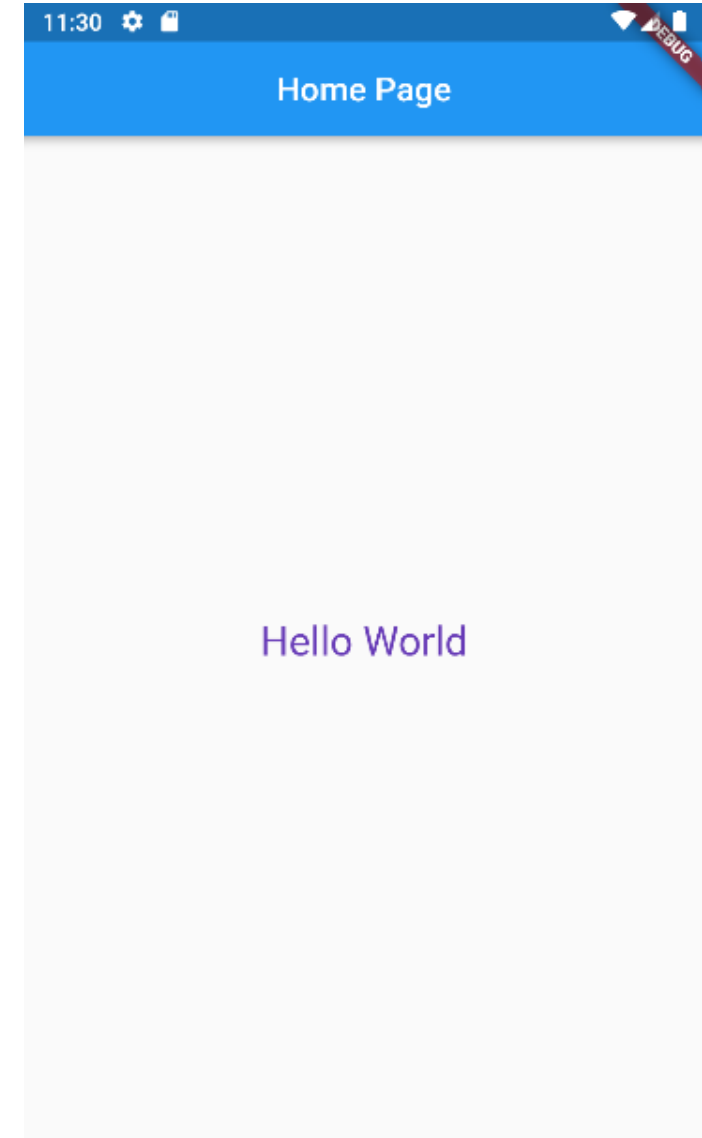
```
@override
Widget build(BuildContext context) {
  return MaterialApp(
    home: Scaffold(
      appBar: AppBar(
        title: Text('Home Page'),
        centerTitle: true,
      ),
      body: Center(
        child: Text('Hello World'),
      ),
    ),
  );
}
```



CHANGING TEXT STYLE

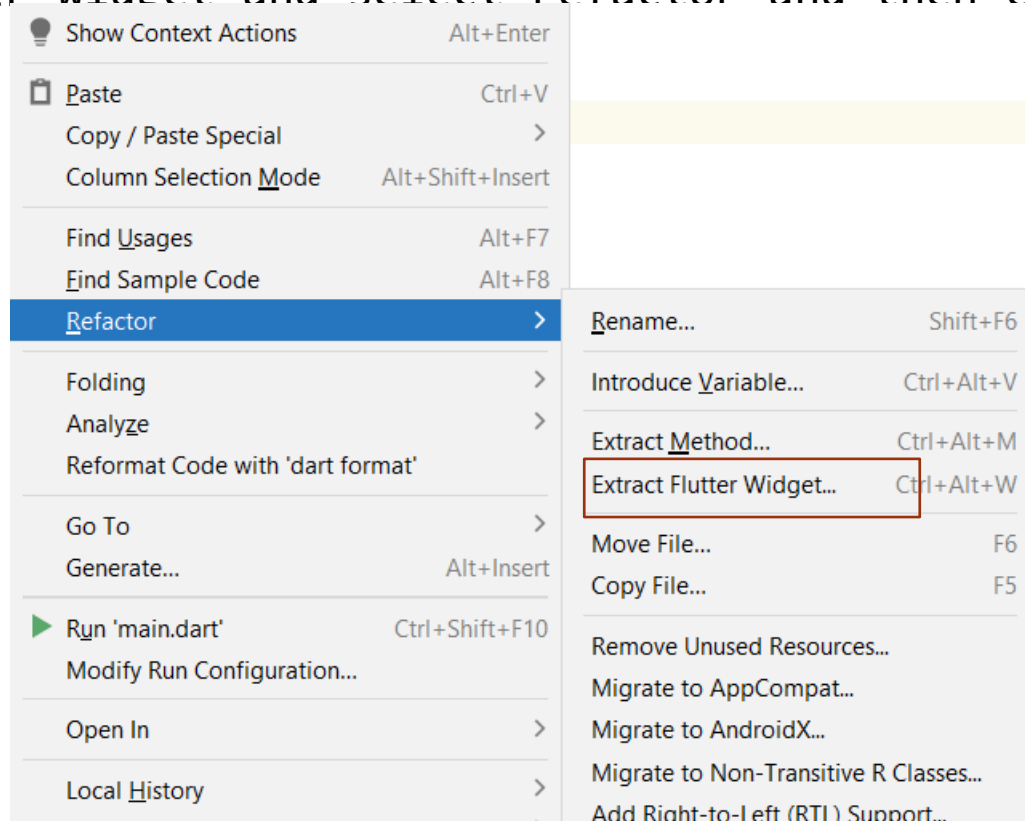
You can change text style by assigning a TextStyle widget to the Text widget's "style" field. Below we define a font size and text color.

```
body: Center(  
  child: Text('Hello World',  
    style: TextStyle(  
      fontSize: 24,  
      color: Colors.deepPurple  
    ),  
  ),  
)
```



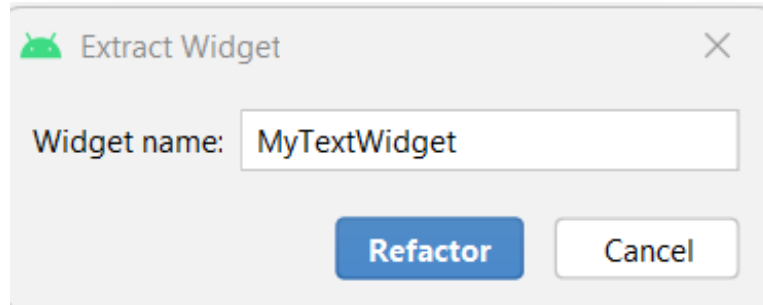
CREATING A CUSTOM WIDGET CLASS

You can convert the Text we created into a custom widget class. This way we can reuse it when we need to create a similar text, elsewhere in our project. Right click on the Center widget and select refactor and then click on extract flutter widget.



CREATING A CUSTOM WIDGET CLASS

Name the widget “MyTextWidget” and then click on the Refactor button.



```
@override
Widget build(BuildContext context) {
  return MaterialApp(
    home: Scaffold(
      appBar: AppBar(
        title: Text('Home Page'),
        centerTitle: true,
      ),
      body: MyTextWidget(),
    ),
  );
}
```

Flutter will replace
the body widget with
the class name.



MODIFY THE MYTEXTWIDGET

Now we will modify the MyTextWidget to accept a text field as parameter. This way we can pass to it the text we want when we call its constructor.

```
class MyTextWidget extends StatelessWidget {  
  final String text; // we add a field called text.  
  // we then add it as a required parameter in the constructor  
  const MyTextWidget({required this.text, super.key});  
  @override  
  Widget build(BuildContext context) {  
    return Center(  
      child: Text(text, // we replace "hello world" text with the text field  
        style: TextStyle(  
          fontSize: 24,  
          color: Colors.deepPurple  
        ),  
    ),  
  );  
}
```

PASSING TEXT TO MYTEXTWIDGET

Now the build method of the MyApp class looks like the below:

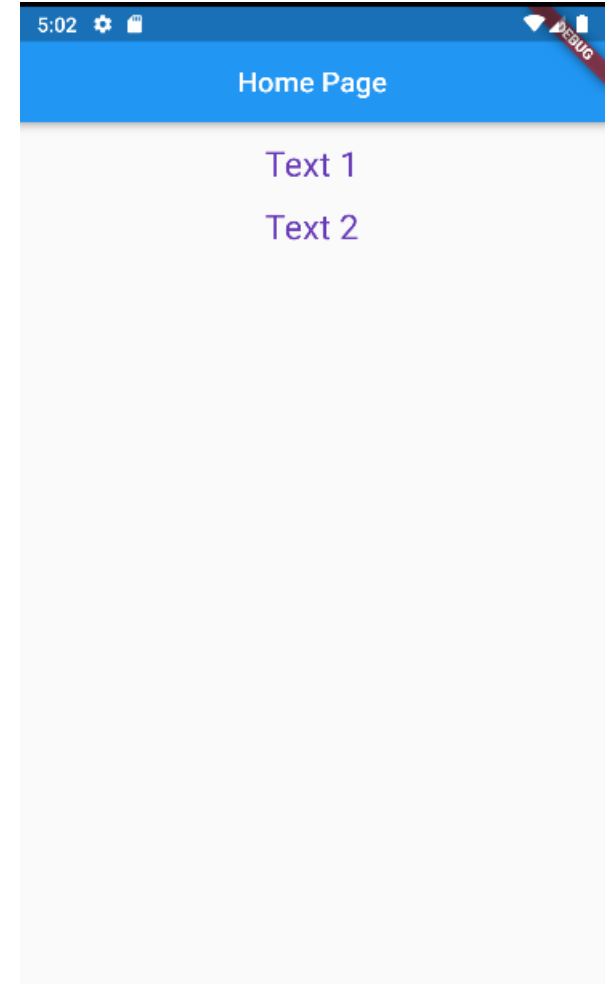
```
@override
Widget build(BuildContext context) {
  return MaterialApp(
    home: Scaffold(
      appBar: AppBar(
        title: Text('Home Page'),
        centerTitle: true,
      ),
      body: MyTextWidget(text: 'Text 1'), // now we can pass any text to the widget
    ),
  );
}
```



ADDING A COLUMN WIDGET

We will use the Column widget, to place 2 Text widgets arranged vertically. You add widgets to the Column as a list of widgets assigned to its “children” field.

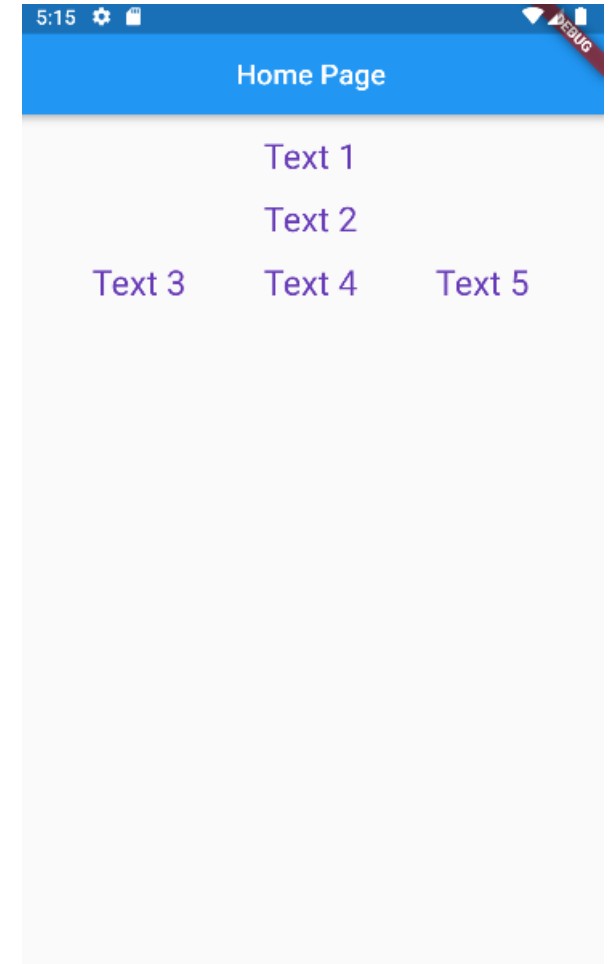
```
body: Column(  
  children: <Widget>[  
    SizedBox(height: 16.0,),  
    MyTextWidget(text: 'Text 1'),  
    SizedBox(height: 16.0,),  
    MyTextWidget(text: 'Text 2')  
  ],  
)
```



ADDING A ROW WIDGET

We will add a row widget to align 3 Text widgets horizontally.

```
body: Column(  
  children: <Widget>[  
    SizedBox(height: 16.0,),  
    MyTextWidget(text: 'Text 1'),  
    SizedBox(height: 16.0,),  
    MyTextWidget(text: 'Text 2'),  
    SizedBox(height: 16.0,),  
    Row(  
      mainAxisAlignment: MainAxisAlignment.spaceEvenly,  
      children: <Widget>[  
        MyTextWidget(text: 'Text 3'),  
        MyTextWidget(text: 'Text 4'),  
        MyTextWidget(text: 'Text 5')  
      ],  
    ),  
  ],  
)
```



LOGICAL PIXELS

- ❑ In the previous slide we wrote `SizeBox(height: 16.0,)`, this creates a vertical space of 16 logical pixels.
- ❑ A logical pixel is the scaling unit in flutter.
- ❑ A logical pixel may convert to multiple physical pixels, depending on the device screen specifications.
- ❑ A logical pixel can be 10 physical pixels on one device and 20 physical pixels on another device.
- ❑ This allows a widget size to remain constant with respect to different screen.

